

Uni-App 基于Promise封装request网络请求工具

在Uni-App开发中，网络请求是与后端交互的核心环节。原生的 `uni.request` 采用回调函数方式处理结果，在复杂业务中易出现“回调地狱”，代码可读性与可维护性较差。基于Promise封装网络请求，可将异步操作同步化表达，配合`async/await`使用，大幅优化代码结构，同时统一处理请求拦截、响应拦截、异常提示等通用逻辑。

一、封装核心思路

Promise封装request的核心是将请求的“成功/失败”结果通过Promise的 `resolve/reject` 方法返回，同时整合以下通用能力，形成高复用的请求工具：

1. **基础配置统一**：集中管理基础域名、默认请求头、超时时间等，便于后期维护；
2. **请求拦截**：发起请求前添加通用逻辑，如携带Token、设置加载状态；
3. **响应拦截**：统一解析响应数据，区分业务错误（如接口返回`code≠200`）与网络错误；
4. **异常统一处理**：网络错误、超时、业务错误等场景统一提示，减少页面重复代码；
5. **个性化配置**：支持传入自定义请求头、关闭加载提示等，适配不同业务场景。

二、完整封装实现（utils/request.js）

在项目 `src/utils` 目录下创建 `request.js` 文件，实现完整的Promise封装，包含GET/POST简化方法及通用拦截逻辑。

Code block

```
1  /**
2   * Uni-App 网络请求工具（基于Promise封装）
3   * 功能：统一请求配置、拦截器、异常处理、加载提示
4   */
5
6  // 基础配置（可根据环境动态切换，如开发/生产环境）
7  const BASE_CONFIG = {
8    baseUrl: process.env.NODE_ENV === 'development'
9      ? 'https://dev-api.example.com' // 开发环境域名
10     : 'https://api.example.com',    // 生产环境域名
11    timeout: 10000, // 超时时间（10秒）
12    defaultHeader: { // 默认请求头
13      'Content-Type': 'application/json'
14    }
15  }
```

```

15 };
16
17 /**
18  * 核心请求方法
19  * @param {Object} options - 请求配置项
20  * @param {String} options.url - 接口路径（相对路径，会拼接baseUrl）
21  * @param {String} [options.method='GET'] - 请求方法（GET/POST/PUT/DELETE等）
22  * @param {Object} [options.data={}] - 请求参数（GET会拼接到URL，POST放在请求体）
23  * @param {Object} [options.header={}] - 自定义请求头（会合并默认请求头）
24  * @param {Boolean} [options.hideLoading=false] - 是否隐藏加载提示
25  * @param {String} [options.loadingText='加载中...'] - 加载提示文本
26  * @returns {Promise} - 返回Promise对象，resolve业务成功数据，reject错误信息
27  */
28 export const request = (options) => {
29   // 1. 合并配置（用户配置覆盖默认配置）
30   const requestOptions = {
31     url: BASE_CONFIG.baseUrl + options.url,
32     method: options.method?.toUpperCase() || 'GET',
33     data: options.data || {},
34     header: { ...BASE_CONFIG.defaultHeader, ...options.header },
35     timeout: BASE_CONFIG.timeout,
36     hideLoading: options.hideLoading || false,
37     loadingText: options.loadingText || '加载中...'
38   };
39
40   // 2. 显示加载提示（默认显示）
41   if (!requestOptions.hideLoading) {
42     uni.showLoading({
43       title: requestOptions.loadingText,
44       mask: true // 遮罩防止点击穿透
45     });
46   }
47
48   // 3. 返回Promise对象，封装请求逻辑
49   return new Promise((resolve, reject) => {
50     // 4. 发起Uni-App原生请求
51     uni.request({
52       ...requestOptions,
53
54       // 5. 请求成功回调（网络层面成功，非业务成功）
55       success: (response) => {
56         // 关闭加载提示
57         if (!requestOptions.hideLoading) {
58           uni.hideLoading();
59         }
60
61         // 解析响应数据（根据后端接口格式调整，此处默认{code, data, msg}）

```

```

62     const { statusCode, data } = response;
63
64     // 网络状态码判断 (200-300为正常)
65     if (statusCode >= 200 && statusCode < 300) {
66         // 业务状态码判断 (假设code=200为业务成功)
67         if (data.code === 200) {
68             // 业务成功, 返回核心数据
69             resolve(data.data);
70         } else {
71             // 业务失败 (如参数错误、权限不足等)
72             const errorMsg = data.msg || '业务请求失败';
73             uni.showToast({ title: errorMsg, icon: 'none', duration: 2000 });
74             reject(new Error(errorMsg));
75         }
76     } else {
77         // 网络状态码异常 (如404、500)
78         const errorMsg = `请求失败[${statusCode}]`;
79         uni.showToast({ title: errorMsg, icon: 'none', duration: 2000 });
80         reject(new Error(errorMsg));
81     }
82 },
83
84 // 6. 请求失败回调 (网络层面失败, 如断网、超时)
85 fail: (error) => {
86     // 关闭加载提示
87     if (!requestOptions.hideLoading) {
88         uni.hideLoading();

```

三、拦截器扩展 (进阶功能)

实际开发中, 常需要在请求发起前添加Token、在响应后处理Token过期等逻辑, 可通过添加“请求拦截器”和“响应拦截器”实现, 使封装更灵活。

Code block

```

1  // 新增拦截器配置
2  const INTERCEPTORS = {
3      // 请求拦截器: 发起请求前执行
4      requestInterceptor: (config) => {
5          // 示例1: 添加Token (从本地缓存获取)
6          const token = uni.getStorageSync('userToken');
7          if (token) {
8              config.header.Authorization = `Bearer ${token}`;
9          }
10
11         // 示例2: GET请求参数处理 (如添加时间戳防缓存)
12         if (config.method === 'GET' && config.data) {

```

```
13     config.data.t = Date.now();
14   }
15
16   return config; // 返回修改后的配置
17 },
18
19 // 响应拦截器: 获取响应后执行 (在业务判断前)
20 responseInterceptor: (response) => {
21   const { data } = response;
22
23   // 示例: Token过期处理 (假设code=401为Token失效)
24   if (data.code === 401) {
25     uni.showToast({ title: '登录已过期, 请重新登录', icon: 'none' });
26     // 跳转至登录页, 清除无效Token
27     uni.clearStorageSync('userToken');
28     uni.navigateTo({ url: '/pages/login/login' });
29   }
30
31   return response; // 返回修改后的响应
32 }
33 };
34
35 // 改造核心request方法, 融入拦截器
36 export const requestWithInterceptor = (options) => {
37   const requestOptions = {
38     url: BASE_CONFIG.baseUrl + options.url,
39     method: options.method?.toUpperCase() || 'GET',
40     data: options.data || {},
41     header: { ...BASE_CONFIG.defaultHeader, ...options.header },
42     timeout: BASE_CONFIG.timeout,
43     hideLoading: options.hideLoading || false,
44     loadingText: options.loadingText || '加载中...'
45   };
46
47   // 执行请求拦截器, 修改请求配置
48   const interceptedConfig = INTERCEPTORS.requestInterceptor(requestOptions);
49
50   if (!interceptedConfig.hideLoading) {
51     uni.showLoading({ title: interceptedConfig.loadingText, mask: true });
52   }
53
54   return new Promise((resolve, reject) => {
55     uni.request({
56       ...interceptedConfig,
57       success: (response) => {
58         if (!interceptedConfig.hideLoading) {
59           uni.hideLoading();
```

```

60     }
61
62     // 执行响应拦截器，处理响应数据
63     const interceptedResponse = INTERCEPTORS.responseInterceptor(response);
64     const { statusCode, data } = interceptedResponse;
65
66     if (statusCode >= 200 && statusCode < 300) {
67         if (data.code === 200) {
68             resolve(data.data);
69         } else {
70             const errorMsg = data.msg || '业务请求失败';
71             uni.showToast({ title: errorMsg, icon: 'none' });
72             reject(new Error(errorMsg));
73         }
74     } else {
75         const errorMsg = `请求失败[${statusCode}]`;
76         uni.showToast({ title: errorMsg, icon: 'none' });
77         reject(new Error(errorMsg));
78     }
79 },
80 fail: (error) => {
81     if (!interceptedConfig.hideLoading) {
82         uni.hideLoading();
83     }
84     let errorMsg = '网络异常，请重试';
85     if (error.errMsg.includes('timeout')) errorMsg = '请求超时';
86     uni.showToast({ title: errorMsg, icon: 'none' });
87     reject(new Error(errorMsg));
88 }

```

四、使用示例（页面/API模块中调用）

封装后的请求工具可在页面中直接调用，配合async/await使用，代码简洁清晰；也可在API模块中进一步封装接口方法，实现“接口分层”。

1. 页面中直接调用（基础用法）

Code block

```

1  <template>
2    <view class="goods-page">
3      <view class="goods-item" v-for="item in goodsList" :key="item.id">
4        <text class="goods-name">{{ item.name }}</text>
5        <text class="goods-price">¥{{ item.price }}</text>
6      </view>
7    </view>
8  </template>
9

```

```
10 <script>
11 import { get, post } from '@utils/request';
12
13 export default {
14   data() {
15     return {
16       goodsList: []
17     };
18   },
19   onLoad() {
20     this.getGoodsList();
21     this.submitForm();
22   },
23   methods: {
24     // 调用GET请求（获取商品列表）
25     async getGoodsList() {
26       try {
27         // 传入接口路径、请求参数、个性化配置（如隐藏加载提示）
28         const data = await get('/goods/list', { categoryId: 1 }, {
29           hideLoading: false,
30           loadingText: '加载商品中...'
31         });
32         this.goodsList = data; // 直接使用业务成功数据
33       } catch (error) {
34         // 捕获异常（可选，若无需单独处理，可依赖工具内置提示）
35         console.error('获取商品列表失败: ', error);
36       }
37     },
38
39     // 调用POST请求（提交表单）
40     async submitForm() {
41       try {
42         const formData = { username: 'test', password: '123456' };
43         const result = await post('/user/login', formData, {
44           // 自定义请求头（覆盖默认）
45           header: { 'Content-Type': 'application/x-www-form-urlencoded' }
46         });
47         // 登录成功，存储Token
48         uni.setStorageSync('userToken', result.token);
49         uni.showToast({ title: '登录成功' });
50       } catch (error) {
51         console.error('登录失败: ', error);
52       }
53     }
54   }
55 }
56 </script>
```

2. API模块封装（推荐用法）

将接口请求统一放在 `src/api` 目录下，实现“页面逻辑”与“接口请求”分离，便于维护和复用。

Code block

```
1  // src/api/goods.js (商品相关接口)
2  import { get, post } from '@utils/request';
3
4  /**
5   * 获取商品列表
6   * @param {Object} params - 请求参数 (categoryId: 分类ID, page: 页码)
7   * @param {Object} options - 个性化配置
8   * @returns {Promise} 商品列表数据
9   */
10 export const getGoodsList = (params, options = {}) => {
11   return get('/goods/list', params, {
12     loadingText: '加载商品中...',
13     ...options
14   });
15 };
16
17 /**
18 * 新增商品
19 * @param {Object} data - 商品信息 (name, price, stock等)
20 * @returns {Promise} 新增结果
21 */
22 export const addGoods = (data) => {
23   return post('/goods/add', data, {
24     loadingText: '提交商品中...'
25   });
26 };
27
28 // src/api/user.js (用户相关接口)
29 import { post, get } from '@utils/request';
30
31 export const userLogin = (formData) => {
32   return post('/user/login', formData);
33 };
34
35 export const getUserInfo = () => {
36   return get('/user/info');
37 };
```

页面中调用API模块：

Code block

```
1 import { getGoodsList } from '@api/goods';
2 import { userLogin } from '@api/user';
3
4 // 调用商品列表接口
5 async getGoods() {
6   try {
7     const data = await getGoodsList({ categoryId: 1, page: 1 });
8     this.goodsList = data;
9   } catch (err) {
10    console.error(err);
11  }
12 }
```

五、关键注意事项

1. 接口格式适配

封装工具中默认后端接口返回格式为 `{ code, data, msg }`，实际开发中需根据后端真实格式调整响应解析逻辑，常见调整点：

- 业务成功码：若后端用 `success: true` 标识成功，需将 `data.code === 200` 改为 `data.success`；
- 数据字段：若核心数据在 `data.result` 中，需将 `resolve(data.data)` 改为 `resolve(data.result)`；
- 错误信息：若错误信息在 `data.errorMessage` 中，需调整 `data.msg` 为对应字段。

2. 跨域问题处理

本地开发时，若前端与后端域名不一致，会出现跨域问题，需在 `manifest.json` 中配置H5跨域代理：

Code block

```
1 // manifest.json -> H5配置
2 "h5": {
3   "devServer": {
4     "proxy": {
5       "/api": {
6         "target": "https://api.example.com", // 后端真实域名
7         "changeOrigin": true, // 开启跨域代理
8         "pathRewrite": { "^/api": "" } // 路径重写（若接口无/api前缀可删除）
9       }
10    }
11  }
```



```
11    }
12  }
```

配置后，需将 `BASE_CONFIG.baseUrl` 改为 `'/api'`（开发环境），实现跨域代理。

3. Token过期与登录态维护

在响应拦截器中处理Token过期逻辑时，需注意避免重复跳转登录页：

Code block

```
1  // 优化Token过期处理（添加跳转锁）
2  let isRedirecting = false; // 防止重复跳转
3  responseInterceptor: (response) => {
4    const { data } = response;
5    if (data.code === 401 && !isRedirecting) {
6      isRedirecting = true; // 上锁
7      uni.showToast({ title: '登录已过期', icon: 'none' });
8      uni.clearStorageSync('userToken');
9      // 延迟跳转，避免Toast被覆盖
10     setTimeout(() => {
11       uni.navigateTo({ url: '/pages/login/login' });
12       isRedirecting = false; // 解锁
13     }, 1500);
14   }
15   return response;
16 }
```

4. 加载提示与用户体验

- 短耗时请求（如获取用户信息）可隐藏加载提示，通过 `hideLoading: true` 配置；
- 加载提示添加 `mask: true`，防止用户在加载时点击页面其他元素，导致逻辑混乱；
- 避免频繁提示：若同一页面同时发起多个请求，可统一控制加载提示（如只显示第一个请求的提示）。

5. 环境切换配置

开发、测试、生产环境的域名不同，可通过 `process.env.NODE_ENV` 自动切换，HBuilderX中需配置环境变量：

Code block

```
1  // 完善基础配置，支持多环境
2  const ENV = {
3    development: {
```

```
4     baseUrl: '/api' // 开发环境 (配合跨域代理)
5   },
6   test: {
7     baseUrl: 'https://test-api.example.com' // 测试环境
8   },
9   production: {
10    baseUrl: 'https://api.example.com' // 生产环境
11  }
12 };
13
14 // 动态获取当前环境 (默认生产环境)
15 const currentEnv = process.env.NODE_ENV || 'production';
16 const BASE_CONFIG = {
17   baseUrl: ENV[currentEnv].baseUrl,
18   timeout: 10000,
19   defaultHeader: { 'Content-Type': 'application/json' }
20 };
```

六、总结

基于Promise封装Uni-App的request网络请求，核心价值在于“统一化、模块化、易维护”：

1. 统一处理通用逻辑，减少页面重复代码；
2. 通过async/await优化异步代码结构，避免回调地狱；
3. 接口分层封装，使页面专注于UI渲染，API模块专注于请求逻辑；
4. 拦截器扩展灵活，可适配Token、权限等复杂业务场景。

实际开发中，可根据项目需求进一步扩展功能，如添加请求取消、重试机制、日志上报等，使请求工具更贴合业务需求。

(注：文档部分内容可能由 AI 生成)